`head` each time through the loop, because the compiler does that for you. Also, because you're just looping directly on failure, you can use `compare_exchange_weak`, which can result in more optimal code than `compare_exchange_strong` on some architectures (see chapter 5).

So, you might not have a `pop()` operation yet, but you can quickly check `push()` against the guidelines. The only place that can throw an exception is the construction of the new `node` ❶, but this will clean up after itself, and the list hasn't been modified yet, so that's perfectly safe. Because you build the data to be stored as part of the `node`, and you use `compare_exchange_weak()` to update the `head` pointer, there are no problematic race conditions here. Once the compare/exchange succeeds, the node is on the list and ready for the taking. There are no locks, so there's no possibility of deadlock, and your `push()` function passes with flying colors.

Of course, now that you have a means of adding data to the stack, you need a way of getting it off again. On the face of it, this is quite simple:

1. Read the current value of `head`.
2. Read `head->next`.
3. Set `head` to `head->next`.
4. Return the `data` from the retrieved `node`.
5. Delete the retrieved node.

However, in the presence of multiple threads, this isn't so simple. If there are two threads removing items from the stack, they both might read the same value of `head` at step 1. If one thread then proceeds all the way through to step 5 before the other gets to step 2, the second thread will be dereferencing a dangling pointer. This is one of the biggest issues in writing lock-free code, so for now you'll just leave out step 5 and leak the nodes.